Negative Sampling in Recommendation: What makes a Good Negative?

MINDS Lab.

Undergraduate Research Intern
HoonUi Lee

2026-09-08

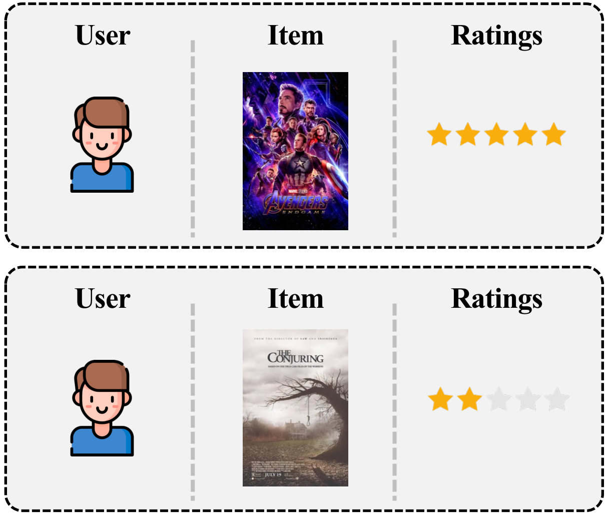
Contents

Recommendation Learns from User Feedback

Why Negative Sampling?

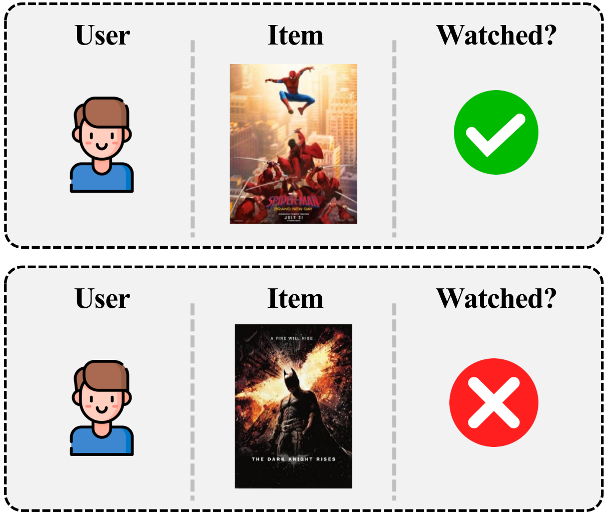
Explicit Feedback

- Users directly express their preferences
  - Rating, Review, Like / Dislike, ...



Implicit Feedback

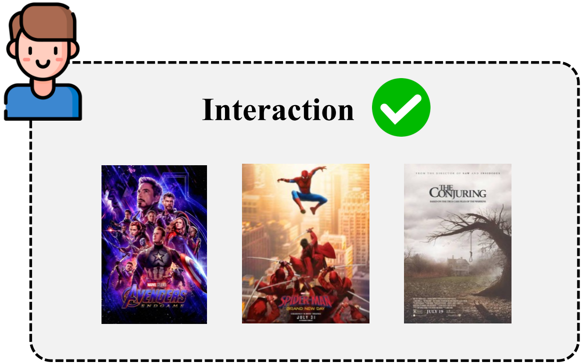
- Preferences are inferred from user behavior
  - Click, Purchase, Watch, Play, ...



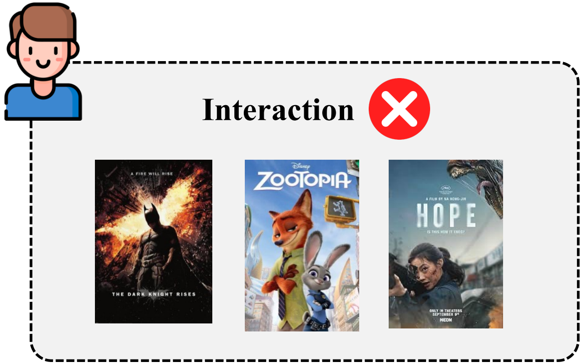
Negative Samples under Implicit Feedback

Why Negative Sampling?

❖ Where Do Negative Samples Come From?



Observed ➞ ***Positive***



Unobserved ➞

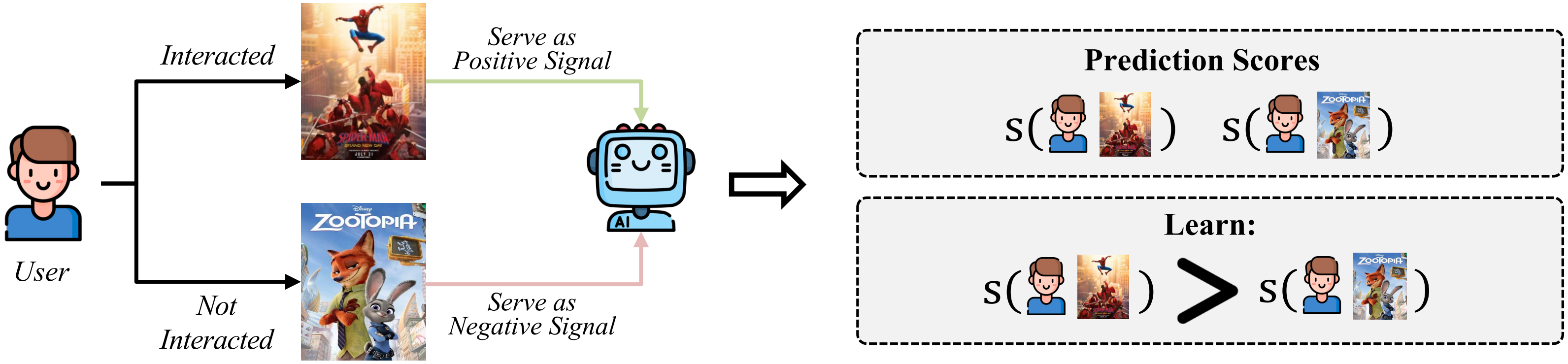
Negative Candidate Pool

- Negative sampling selects a subset of training negatives from the massive unobserved item pool

How Are Positive and Negative Samples Used in Training?

Why Negative Sampling?

❖ Learning Relative Preference from Positive and Negative Samples

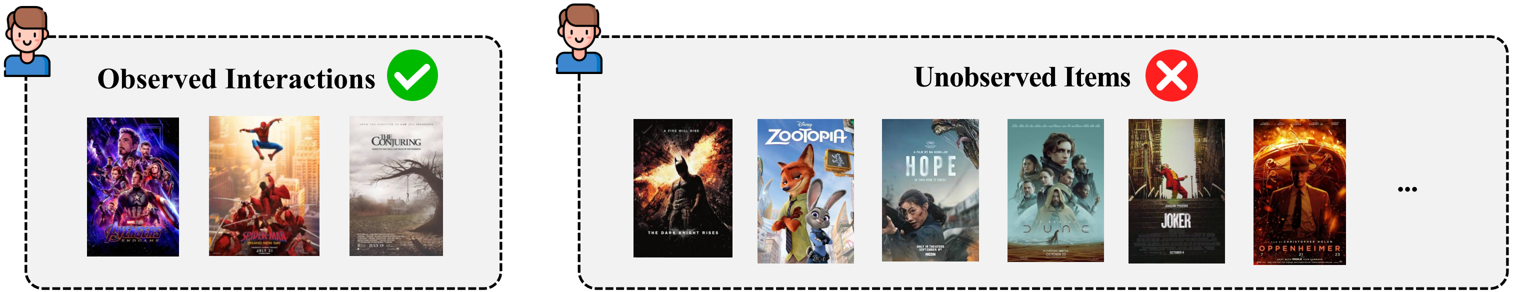


- ❑ Positive–Negative Comparison
  - ▸ Preference scores for positive and sampled negative items
- ❑ Relative Preference Optimization
  - ▸ Higher score for positives than sampled negatives

Why Do We Need Sampling?

Why Negative Sampling?

❖ Too Many Negative Candidates for Training



❑ **Large-scale Recommendation**
- Millions of users & items
- Massive item corpus

❑ **Sparse User Interactions**
- Few observed interactions per user
- Most user–item pairs remain unobserved

❑ **Computational Burden**
- Full-corpus training is impractical
- High cost per training update

Uniform Negative Sampling

Why Negative Sampling?

❖ The Simplest Sampling Strategy

- ❑ Model-Agnostic Selection
  - No consideration of current model predictions
- ❑ Simple and Efficient
  - No additional scoring or side information

From Hard Negatives to Better Negatives

Why Negative Sampling?

❖ How the definition of a "good negative" evolves

- ❏ DNS — Find Hard Negatives
  - **Good Negative = Hardness**
- ❏ SRNS — Make Hard Negatives Reliable
  - **Good Negative = Hardness + Reliable**
- ❏ DNS+ — Control the Right Hardness
  - **Good Negative = Appropriately Hardness**

Optimizing Top-N Collaborative Filtering via Dynamic Negative Item Sampling

Weinan Zhang
Dept. of Computer Science, University College London

Tianqi Chen
Dept. of Computer Science and Engineering, Shanghai Jiao Tong University

Jun Wan
Dept. of Computer Science, University College London

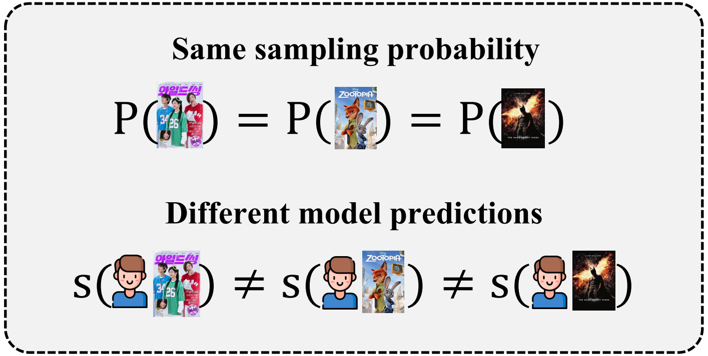
Yong Yu
Dept. of Computer Science and Engineering, Shanghai Jiao Tong University

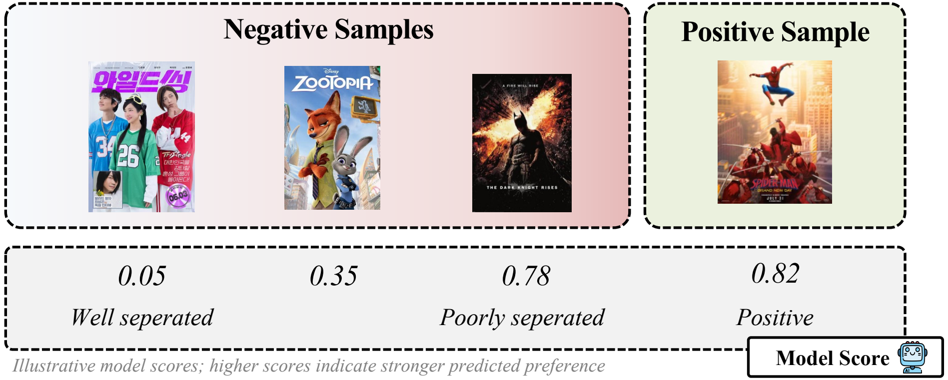
Published in SIGIR'13

Do All Negatives Look the Same to the Current Model?

Why Negative Sampling?

❖ Same Sampling Rule, Different Model Responses





Illustrative model scores; higher scores indicate stronger predicted preference

- Different Levels of Separation
  - Same sampling probability, but different score gaps from the positive

➔ **Does this difference matter during training?**

Why Does the Negative Score Matter?

Why Negative Sampling?

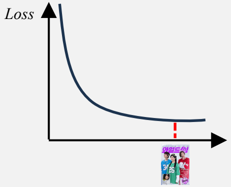
❖ Different Score Gaps, Different Optimization Effects

$\mathcal{L}_{BPR} = -\log\sigma\left(s(u,i^+) - s(u,j^-)\right),\quad s(u,i^+):\ 0.82$ 

Well-separated negative

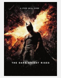
$s(u,j_1) = 0.05$

$\Delta_1 = 0.82 - 0.05 = 0.77$



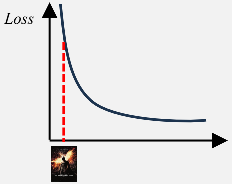
Smaller gradient magnitude

Poorly-sepearated negative



$s(u,j_2) = 0.78$

$\Delta_2 = 0.82 - 0.78 = 0.04$



Larger gradient magnitude

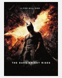
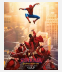
- ❑ Already well-separated negatives provide less additional optimization pressure
- ❑ The usefulness of a negative depends on the current model state

Rank Matters

Why Negative Sampling?

❖ **Higher-ranked negatives have a larger impact on rank-biased metrics**

Current Ranking

Rank		*Item*	*Score*
Rank 1		*Negative j_1*	0.85
Rank 2		*Negative j_2*	0.72
Rank 3		*Negative j_3*	0.58
Rank 4		*Positive i*	0.55

$NDCG@4 = 0.431$

High Score ➔ High-Ranked Error

What If This Pair Were Corrected?

Swap i and j_1

$NDCG@4 = 1.000 \Rightarrow \Delta NDCG@4 = 1.000 - 0.431 = \mathbf{0.569}$ **(Large ranking impact)**

Swap i and j_3

$NDCG@4 = 0.500 \Rightarrow \Delta NDCG@4 = 0.500 - 0.431 = \mathbf{0.069}$ **(Small ranking impact)**

Higher Rank ➔ Larger Potential Ranking Impact

❑ Higher-ranked negatives should receive more training effort

Weight What Matters

Why Negative Sampling?

❖ From ranking impact to learning priority

BPR

$$\frac{\partial L_{ij}}{\partial w} = \frac{\partial L_{ij}}{\partial (s_{ui} - s_{uj})}\left(\frac{\partial s_{ui}}{\partial w} - \frac{\partial s_{uj}}{\partial w}\right)$$

$$= \lambda_{ij}\left(\frac{\partial s_{ui}}{\partial w} - \frac{\partial s_{uj}}{\partial w}\right)$$

Pairwise Gradient Signal: λ_{ij}

"How difficult is this pair for current model?"

⇨

Rank-aware Weighting

Rank-aware Gradient Scale: $\lambda_{ij} \cdot \Delta NDCG_{ij}$

"How important is this error?"

$\Delta NDCG@4 = \mathbf{0.569}$ ➔ ***Higher Priority***

$\Delta NDCG@4 = \mathbf{0.069}$ ➔ ***Lower Priority***

❑ But full ranking is expensive

- Computing $\Delta NDCG$ requires the current ranking over all unseen items

➔ **Can we get the same effect through sampling instead of explicit weighting?**

Sample What Matters

Why Negative Sampling?

❖ From rank-aware weighting to rank-aware sampling

Sampling Instead of Weighting

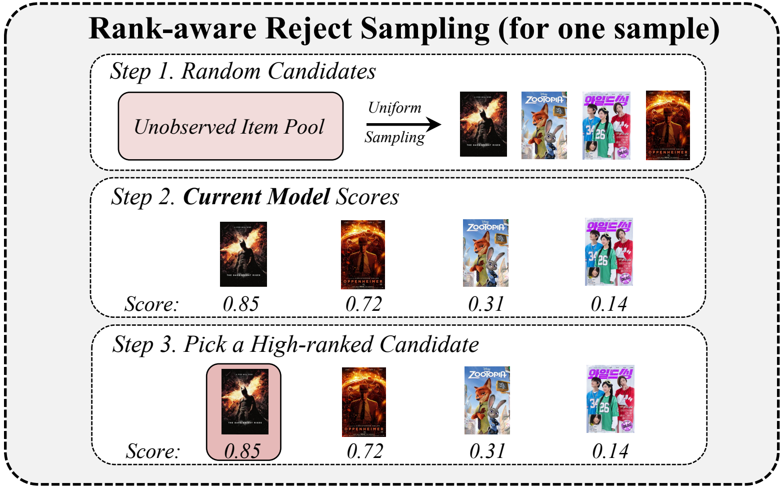
$$p_j \propto \frac{f(\lambda_{ij}, \zeta_u)}{\lambda_{ij}}, f(\lambda_{ij}, \zeta_u) = \lambda_{ij} \cdot \Delta NDCG_{ij}$$

$$\rightarrow p_j \propto \Delta NDCG_{ij}$$

Larger ranking impact → Higher sampling probability

But ΔNDCG itself still requires full ranking...

⇨

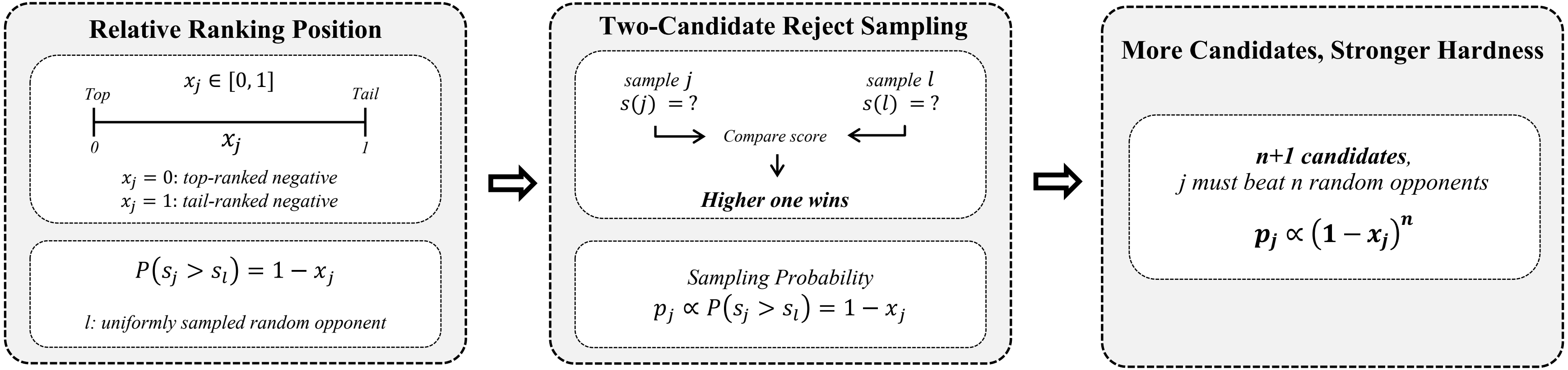


- ❑ Sample a few → Score them → Prefer higher-ranked negatives
  - Dynamic: the hard negative changes as the model changes

Sample What Matters (Cont.)

Why Negative Sampling?

❖ Why does reject sampling become rank-aware?



- ❑ Higher-ranked negatives win random comparisons more often ➔ Higher Sampling Probability
- ❑ Random competition implicitly approximates a rank-biased sampling distribution

Does DNS Improve Learning?

❖ DNS improves both ranking quality and training efficiency

Ranking Performance

Netflix

	P@5	P@10	NDCG@5	NDCG@10	MAP
BPR	0.3826	0.3272	0.2052	0.2017	0.1403
DNS	0.4708	0.4012	0.2906	0.2887	0.2036
Impv.	23.1%*	22.6%*	41.6%*	43.1%*	45.1%*

Yahoo! Music

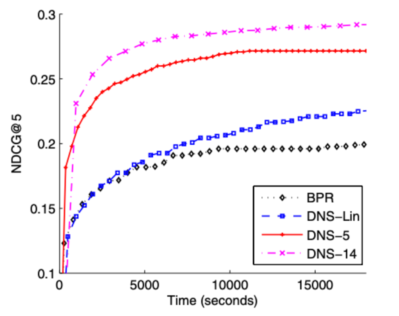
	P@5	P@10	NDCG@5	NDCG@10	MAP
BPR	0.1588	0.1359	0.1683	0.1481	0.0615
DNS	0.4243	0.3671	0.4458	0.3981	0.1644
Impv.	167.2%*	170.1%*	164.9%*	168.8%*	167.3%*

Last.fm

	P@5	P@10	NDCG@5	NDCG@10	MAP
BPR	0.1231	0.1168	0.1270	0.1207	0.0221
DNS	0.1323	0.1202	0.1355	0.1250	0.0223
Impv.	7.5%*	2.9%	6.7%*	3.6%	0.9%

Harder negatives substantially improve rank-biased metrics

Training Efficiency



More informative updates require fewer training rounds

- ❑ Current model score provides a useful signal for negative selection
- ❑ Sampling informative negatives improves both ranking quality and convergence

Simplify and Robustify Negative Sampling for Implicit Collaborative Filtering

Jingtao Ding
Tsinghua University

Yuhan Quan
Tsinghua University

Quanming Yao
4Paradigm Inc (Hong Kong)

Yong Li
Tsinghua University

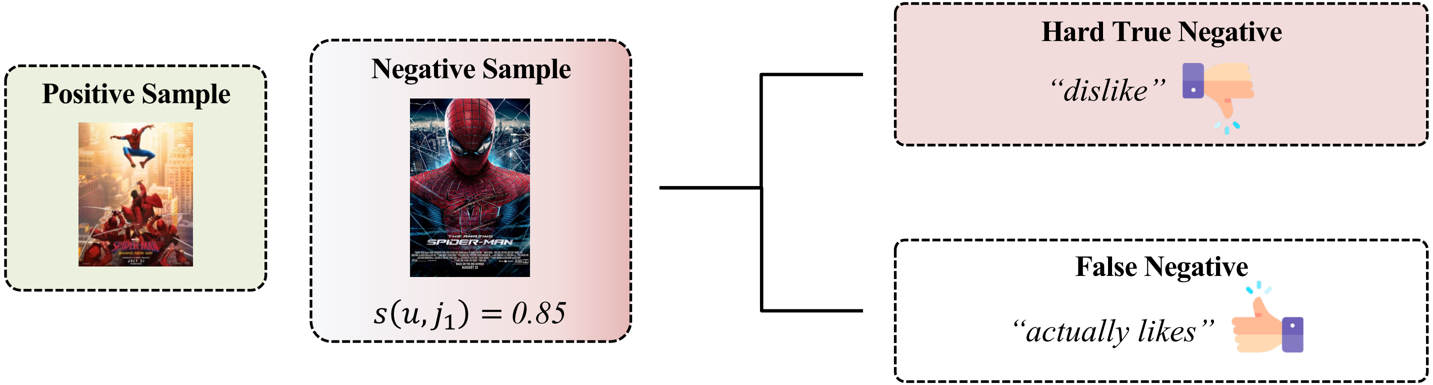
Depeng Jin
Tsinghua University

Published in NeurIPS'20

Is Hard Always Good?

Why Negative Sampling?

❖ **Hard negative sampling may amplify false negatives**



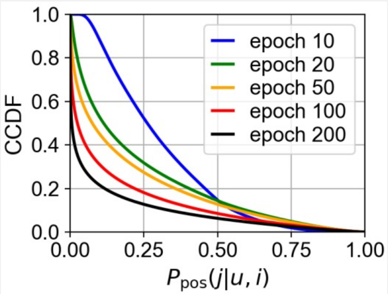
- An item may be unobserved because of exposure, position, or other factors — not necessarily dislike
  - High Score ≠ True Negative
- Can we distinguish hard true negatives from false negatives?

Understanding False Negatives

Why Negative Sampling?

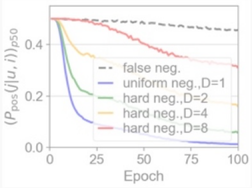
❖ Finding 1: Hard negatives are rare

Only a Few Negatives Are Hard



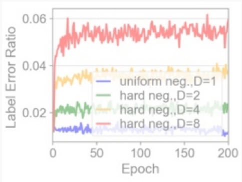
$$F(x) = P(P_{pos} \geq x)$$

False Negatives Also Look Hard



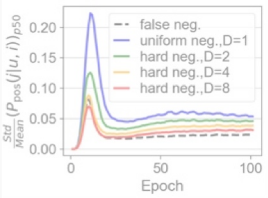
D: number of random candidates used to select one negative

Harder Sampling Selects More False Negatives



$$\text{Label Error Ratio} = \frac{\#\,\text{selected false negatives}}{\#\,\text{selected negatives}}$$

False Negatives Are More Stable



$$\text{Normalized Variance} = \frac{Std(P_{pos})}{Mean(P_{pos})}$$

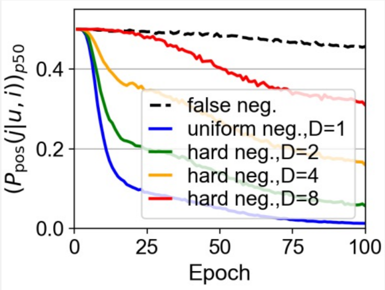
❑ Only a small fraction of unobserved items have high positive-label probability P_{pos}

Understanding False Negatives

Why Negative Sampling?

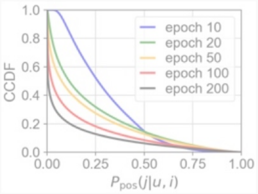
❖ Finding 2: Score captures hardness, but not reliability

False Negatives Also Look Hard



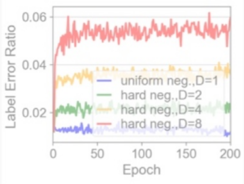
D: number of random candidates used to select one negative

Only a Few Negatives Are Hard



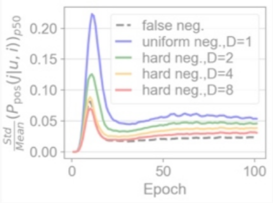
$F(x) = P(P_{pos} \geq x)$

Harder Sampling Selects More False Negatives



$\text{Label Error Ratio} = \frac{\#\,\text{selected false negatives}}{\#\,\text{selected negatives}}$

False Negatives Are More Stable



$\text{Normalized Variance} = \frac{Std(P_{pos})}{Mean(P_{pos})}$

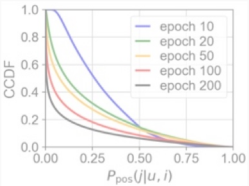
- ❑ Harder negatives have larger P_{pos}, but false negatives also maintain high scores
- ❑ Prediction score alone cannot distinguish hard true negatives from false negatives

Understanding False Negatives

Why Negative Sampling?

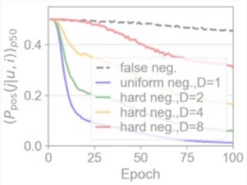
❖ **Harder is more informative, but also noisier**

Only a Few Negatives Are Hard



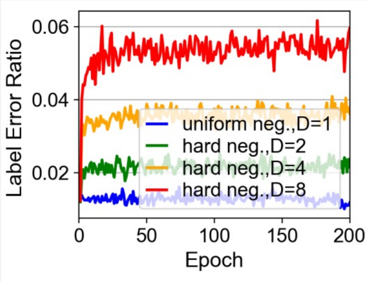
$F(x) = P(P_{pos} \geq x)$

False Negatives Also Look Hard



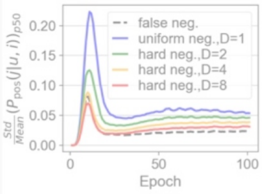
D: number of random candidates used to select one negative

Harder Sampling Selects More False Negatives



$$Label\ Error\ Ratio = \frac{\#\ selected\ false\ negatives}{\#\ selected\ negatives}$$

False Negatives Are More Stable



$$Normalized\ Variance = \frac{Std(P_{pos})}{Mean(P_{pos})}$$

❑ As D increases, the Label Error Ratio consistently increases

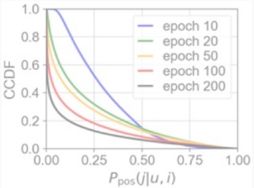
- Aggressive hard-negative mining increases false-negative contamination

Understanding False Negatives

Why Negative Sampling?

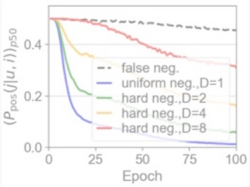
❖ **Prediction variance provides an additional reliability signal**

Only a Few Negatives Are Hard



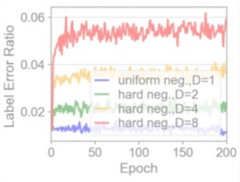
$F(x) = P(P_{pos} \geq x)$

False Negatives Also Look Hard



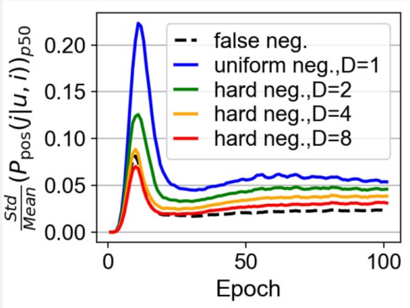
D: number of random candidates used to select one negative

Harder Sampling Selects More False Negatives



$\text{Label Error Ratio} = \frac{\#\,\text{selected false negatives}}{\#\,\text{selected negatives}}$

False Negatives Are More Stable



$$\text{Normalized Variance} = \frac{Std(P_{pos})}{Mean(P_{pos})}$$

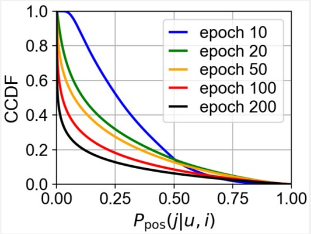
❑ False negatives show lower prediction variance than true negative samples

- Their predictions remain relatively stable throughout training

Understanding False Negatives (Cont.)

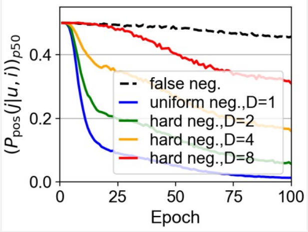
Why Negative Sampling?

Only a Few Negatives Are Hard



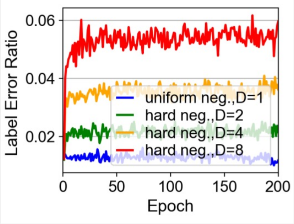
$F(x) = P(P_{pos} \geq x)$

False Negatives Also Look Hard



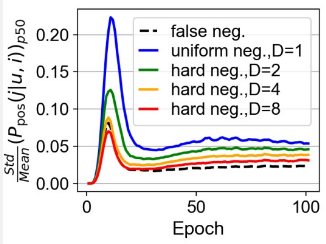
D: number of random candidates used to select one negative

Harder Sampling Selects More False Negatives



$\text{Label Error Ratio} = \frac{\#\ selected\ false\ negatives}{\#\ selected\ negatives}$

False Negatives Are More Stable



$\text{Normalized Variance} = \frac{Std(P_{pos})}{Mean(P_{pos})}$

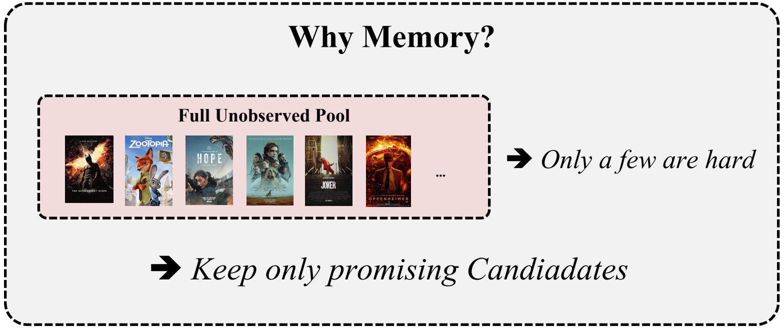
Current Score ➔ **Hardness**

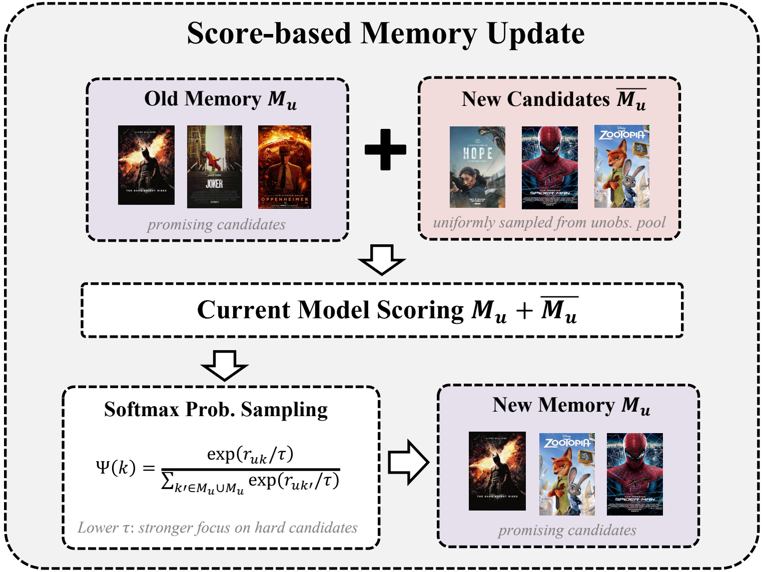
Prediction History ➔ **Reliability**

Remember the Hard Ones

Why Negative Sampling?

❖ Maintain a small memory of promising negatives



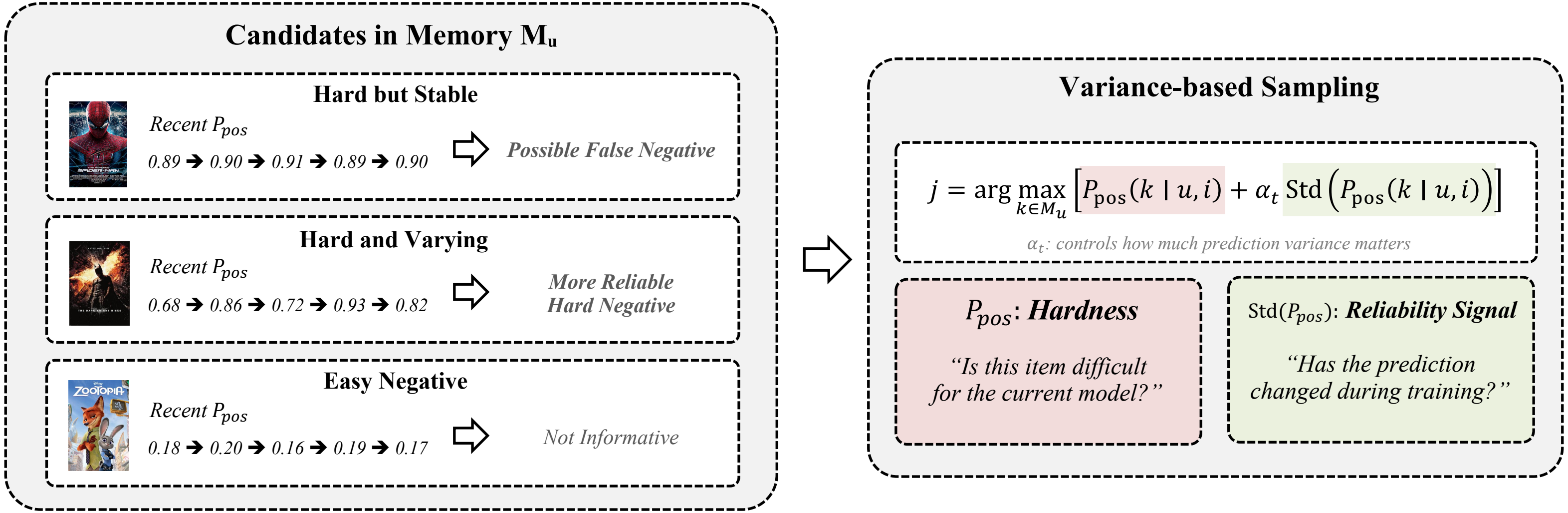


- ❑ Remember promising negatives instead of rediscovering them from the full pool
- ❑ **But high-score memory can still contain false negatives. How should we choose one reliably?**

Hard, But Reliable

Why Negative Sampling?

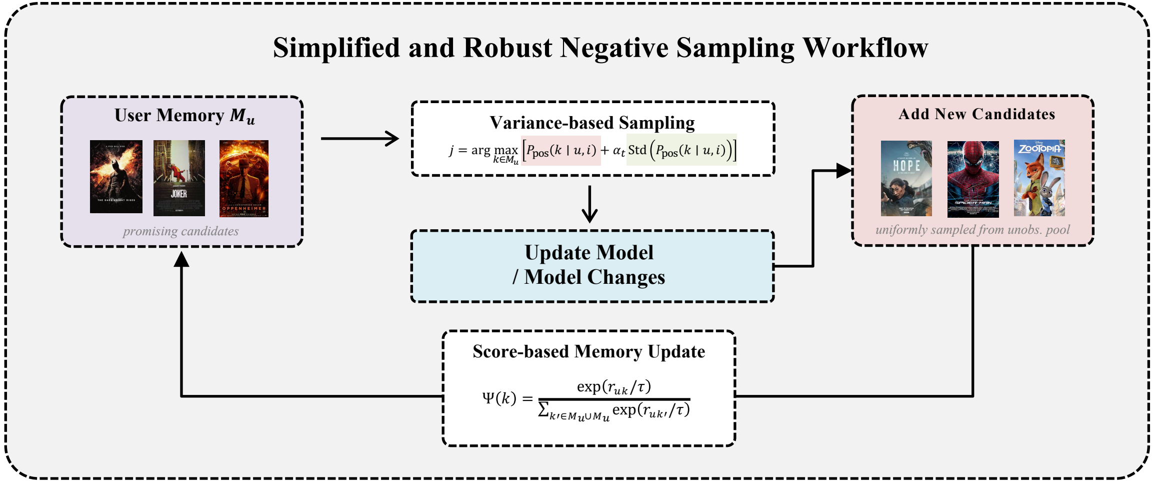
❖ Select negatives using both hardness and prediction stability



❑ Do not simply choose the hardest negative — prefer one that is hard and less likely to be false.

Putting SRNS Together

- Alternating between robust sampling and dynamic memory update



Warm Start

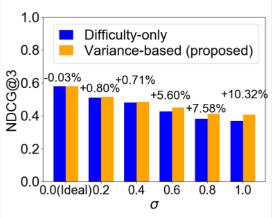
$$\alpha_t = \alpha \min\left(\frac{t}{T_0}, 1\right)$$

Early: *Hardness first*
Later: *Hardness + Reliability*

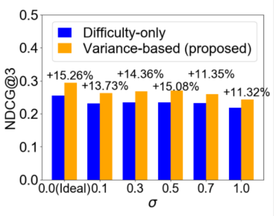
Does Reliability Help?

- Variance-aware sampling improves robustness and recommendation quality

Robustness to False Negatives



(a) N@3, ML-100k



(c) N@3, Ecom-toy

Variance-based sampling remains more robust as false-negative noise increases

Real-world Performance

Method	Movielens-1m			Pinterest			Ecommerce		
	N@1	N@3	R@3	N@1	N@3	R@3	N@1	N@3	R@3
ENMF	0.1846	0.3021	0.3882	0.2594	0.4144	0.5284	0.1317	0.2095	0.2670
Uniform	0.1744	0.2846	0.3663	0.2586	0.4136	0.5276	0.1265	0.2057	0.2640
NNCF	0.0829	0.1478	0.1971	0.2292	0.3699	0.4735	0.0833	0.1420	0.1855
AOBPR	0.1802	0.2905	0.3728	0.2596	0.4165	0.5319	0.1293	0.2108	0.2710
IRGAN	0.1755	0.2877	0.3708	0.2587	0.4143	0.5282	0.1275	0.2065	0.2648
RNS-AS	0.1823	0.2932	0.3754	0.2690	0.4233	0.5359	0.1335	0.2131	0.2714
AdvIR	0.1790	0.2941	0.3792	0.2689	0.4235	0.5363	0.1357	0.2141	0.2719
SRNS	**0.1933**	**0.3070**	**0.3912**	**0.2891**	**0.4391**	**0.5486**	**0.1471**	**0.2256**	**0.2833**
	4.71%	1.62%	0.77%	7.47%	3.68%	2.29%	8.40%	5.37%	4.19%

- Current score finds informative negatives, Prediction history reduces the risk of false negatives

On the Theories Behind Hard Negative Sampling for Recommendation

Wentao Shi
University of Science and Technology of China

Jiawei Chen
Zhejiang University

Fuli Feng
University of Science and Technology of China

Jizhi Zhang
University of Science and Technology of China

Junkang Wu
University of Science and Technology of China

Chongming Gao
University of Science and Technology of China

Xiangnan He
University of Science and Technology of China

Published in WWW'23

Beyond Faster Convergence

Why does Hard Negative Sampling improve recommendation accuracy?

❖ Is faster convergence enough to explain HNS?

Conventional view

Hard Negative ➔ Larger Gradient
➔ Faster Convergence

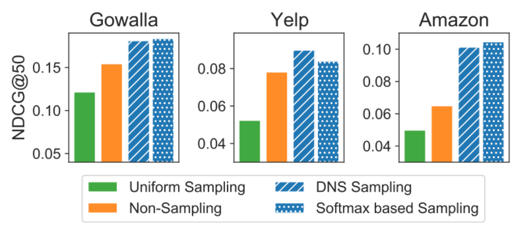
This explains training efficiency —
but does it explain better final accuracy?

BPR Loss with Sampling Distribution

$$\min_{\theta} \sum_{c \in C} \sum_{i \in \mathcal{I}_c^+} E_{j \sim P_{ns}(j|c)} \left[\ell \left(r(c,i|\theta) - r(c,j|\theta) \right) \right]$$

Negative Sampling Distribution changes
the effective training objective

An Unexpected Observation



HNS > Non-Sampling > Uniform

*But Non-sampling uses **ALL** negatives*

❑ HNS may change not only how fast we optimize, but what we optimize

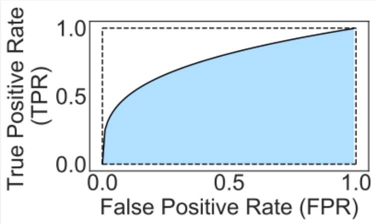
➔ **What does Hard Negative Sampling actually optimize?**

Focus on the Top

Why does Hard Negative Sampling improve recommendation accuracy?

❖ Top-K recommendation does not care about every ranking error equally

AUC: Evaluate the Entire Ranking

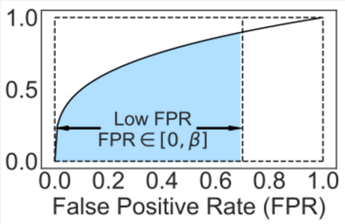


$$AUC = P\big(s(u,i^+) > s(u,j^-)\big)$$

A randomly chosen positive should outrank a randomly chosen negative

Recommendation is evaluated at the top of the list →

OPAUC: Focus on the Top



$$OPAUC(\beta) = \text{AUC over FPR} \in [0, \beta]$$

$(OPAUC(1) = AUC)$

Low FPR corresponds to a high score threshold
→ Only the highest-scored negatives are considered

- ❑ AUC considers the whole negative population, while OPAUC focuses on top-ranked high-scored negatives

Why Does HNS Optimize OPAUC?

Why does Hard Negative Sampling improve recommendation accuracy?

❖ Both DNS and OPAUC focus on the same top-ranked negative region

Negative Items Ranked by Model Score

Hard / High Score ← — *Easy / Low Score*

j_1 j_2 j_3 … j_M | j_{M+1} j_n

Top Region

DNS(M)

Train on the Top-M Negatives

$$P_{DNS}(j) = \begin{cases} \frac{1}{M}, & j \in \text{Top}-M \\ 0, & \text{otherwise} \end{cases}$$

$M \downarrow \rightarrow$ Narrower top region $\rightarrow$ Harder sampling

↔ $M = \beta n^{-}$

Same Region

OPAUC(β)

Evaluate the Top-β Negative Region

Top-β Fraction = βn^{-} negatives

$\beta \downarrow \rightarrow$ Narrower top region $\rightarrow$ More top-focused

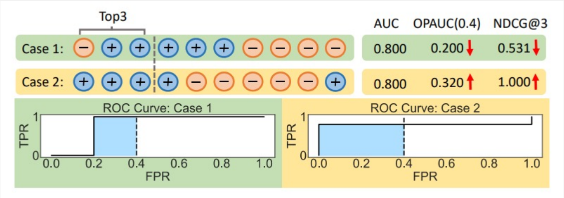
- ❑ BPR + DNS ≡ OPAUC(β) surrogate (Formal proof via CVaR-DRO)
- ❑ DNS shifts learning toward the ranking region that matters most at the top

Why OPAUC Matches Top-K

Why does Hard Negative Sampling improve recommendation accuracy?

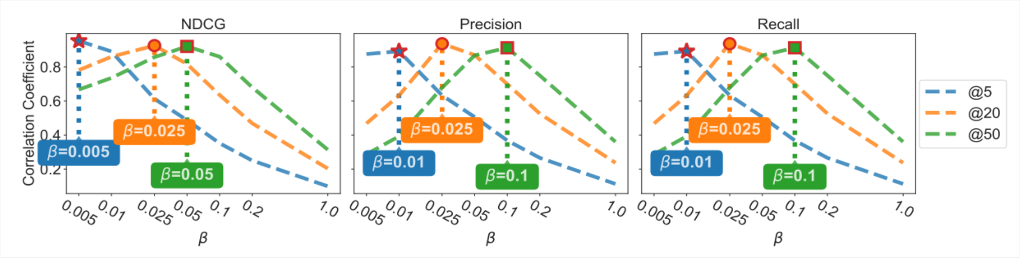
❖ The same overall ranking quality can produce very different Top-K results

Same AUC, Different Top-K



AUC may miss where ranking errors occur, while OPAUC is sensitive to errors near the top

Correlation with Top-K Metrics



OPAUC achieves stronger correlation with NDCG, Precision, and Recall than AUC

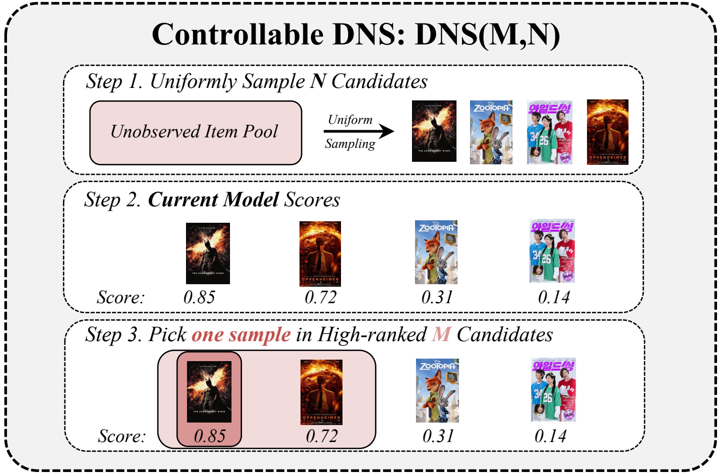
$K\downarrow \rightarrow \beta\downarrow$
Smaller K favors more top-focused objective

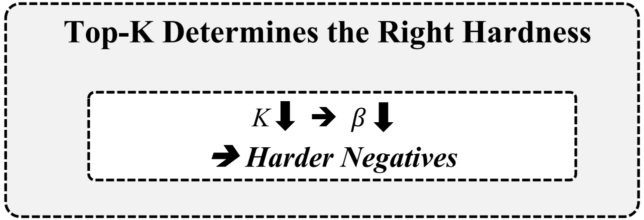
- ❑ AUC measures global pairwise ranking quality, regardless of where the errors occur
- ❑ OPAUC emphasizes errors among high-ranked negatives
  - ▸ which are exactly the errors that affect Top-K recommendation
- ❑ **As the target K becomes smaller, the optimal OPAUC region also becomes narrower**

How Hard Should It Be?

Why does Hard Negative Sampling improve recommendation accuracy?

❖ **Optimal hardness depends on the target Top-K metric**





- ❑ Hardness should be tuned to the ranking region emphasized by the target metric
- ❑ Smaller K focuses evaluation closer to the top, requiring harder negative samples

Does Controlled Hardness Help?

Why does Hard Negative Sampling improve recommendation accuracy?

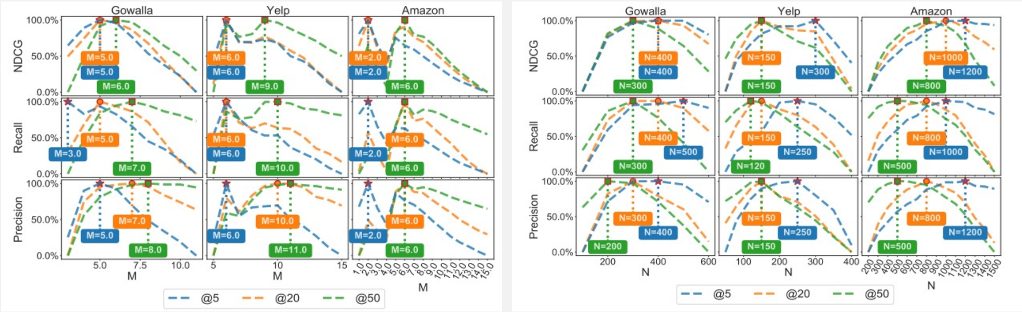
❖ Controllable hardness improves performance and follows the Top-K guideline

Controllable HNS Improves Performance

Method	Gowalla		Yelp		Amazon	
	NDCG@50	Recall@50	NDCG@50	Recall@50	NDCG@50	Recall@50
BPR	0.1216	0.2048	0.0524	0.1083	0.0499	0.1171
AOBPR	0.1385	0.2417	0.0677	0.1346	0.0563	0.1303
WARP	0.1248	0.2240	0.0636	0.1332	0.0542	0.1267
IRGAN	0.1443	0.2242	0.0695	0.1367	0.0627	0.1395
Kernel	0.1399	0.2264	0.0658	0.1315	0.0700	0.1495
DNS	0.1412	0.1839	0.0693	0.1425	0.0615	0.1378
PRIS(U)	0.1334	0.2217	0.0639	0.1273	0.0607	0.1377
PRIS(P)	0.1385	0.2282	0.0673	0.1342	0.0697	0.1463
AdaSIR(U)	0.1489	0.2500	0.0732	0.1523	0.0731	0.1505
AdaSIR(P)	0.1519	0.2516	0.0731	0.1525	0.0740	0.1534
DNS(M, N)	0.1811**	0.2989**	**0.0899****	**0.1774****	0.1014**	0.1833**
Softmax-v(ρ, N)	**0.1837****	**0.2993****	0.0840**	0.1690**	**0.1046****	**0.1937****

Controlling hardness consistently outperforms fixed hard-negative sampling

Smaller K Prefers Harder Negatives

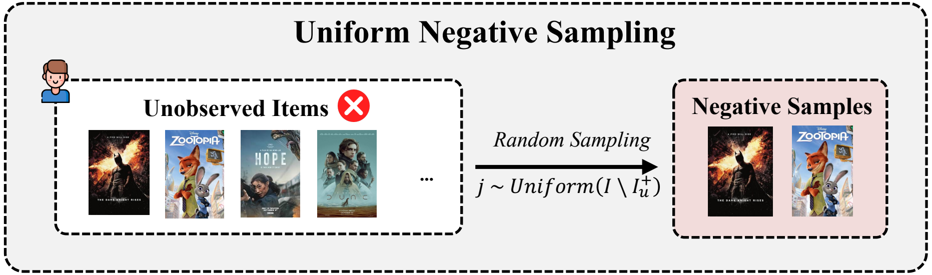


$K\downarrow \rightarrow M^*\downarrow, N^*\uparrow$

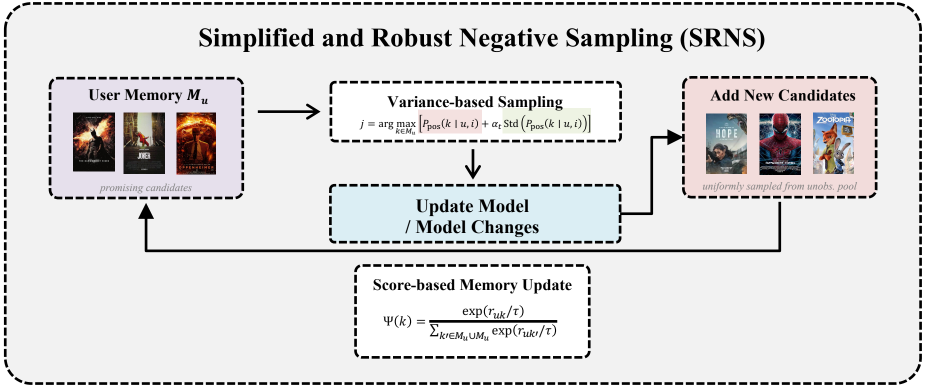
➔ Harder Sampling

❑ Hardness should be controlled according to the ranking region emphasized by the target metric

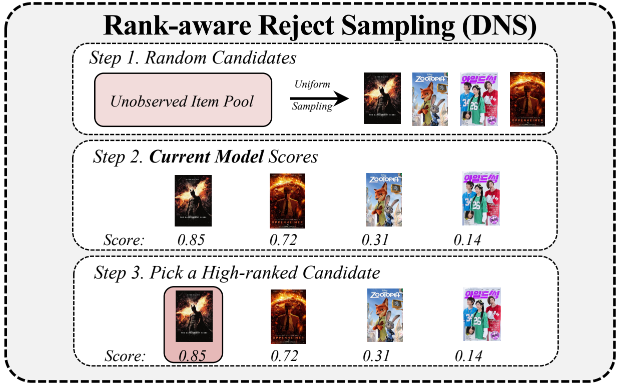
Conclusion



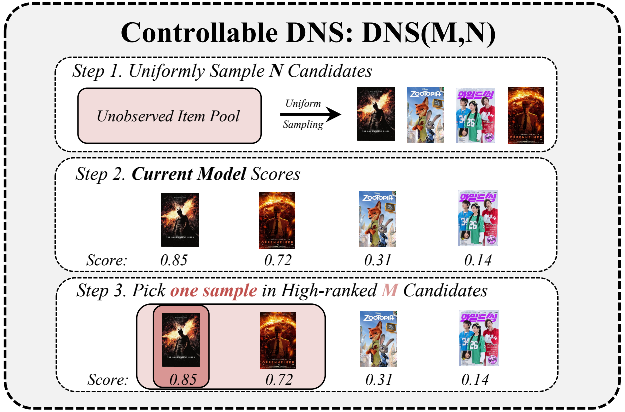
Uniform Negative Sampling
Unobserved Items
...
Random Sampling
$j \sim Uniform(I \setminus I_u^+)$
Negative Samples



Simplified and Robust Negative Sampling (SRNS)
User Memory M_u
promising candidates
Variance-based Sampling
$j = \arg\max_{k \in M_u} [P_{pos}(k \mid u,i) + \alpha_t \mathrm{Std}(P_{pos}(k \mid u,i))]$
Add New Candidates
uniformly sampled from unobs. pool
Update Model / Model Changes
Score-based Memory Update
$\Psi(k) = \frac{\exp(r_{uk}/\tau)}{\sum_{k' \in M_u \cup M_u} \exp(r_{uk'}/\tau)}$



Rank-aware Reject Sampling (DNS)
Step 1. Random Candidates
Unobserved Item Pool
Uniform Sampling
Step 2. Current Model Scores
Score: 0.85 0.72 0.31 0.14
Step 3. Pick a High-ranked Candidate
Score: 0.85 0.72 0.31 0.14



Controllable DNS: DNS(M,N)
Step 1. Uniformly Sample N Candidates
Unobserved Item Pool
Uniform Sampling
Step 2. Current Model Scores
Score: 0.85 0.72 0.31 0.14
Step 3. Pick one sample in High-ranked M Candidates
Score: 0.85 0.72 0.31 0.14


THANKS